

EKF con gestione degli outliers

Tommaso Raffaelli 234813

2026-06-16

Contents

1	Introduzione	2
2	SLAM	2
3	Filtro di Kalman	2
3.1	Filtro di Kalman esteso	3
3.2	Modello Cinematico dell'Uniciclo	4
	Definizione dello stato	4
	Modello cinematico	4
4	Modelli dei Sensori	5
4.1	IMU Planare	5
	Modello del rumore e implementazione	5
	Problemi con sensori propriocettivi	5
4.2	Telecamera (Modello Pinhole)	6
	Funzionamento e Vincoli Visivi	6
	Proiezione e Utilizzo nell'EKF (Fase di Aggiornamento)	7
4.3	Gestione degli Outliers	7
	La Distanza di Mahalanobis	7
4.4	Il Test del Chi-Quadro (χ^2)	8

1 Introduzione

Nel campo della robotica mobile, in particolare per quanto riguarda i veicoli autonomi, esistono due problemi fondamentali e complementari:

- *Localizzazione*, la mappa dell'ambiente è già nota e l'obiettivo è determinare la posizione esatta e l'orientamento del veicolo all'interno di essa.
- *Mappatura*, la posizione del veicolo è nota a priori e l'obiettivo è esplorare l'ambiente per ricostruirne una mappa accurata a partire dai dati dei sensori.

Questi due processi sono, essenzialmente, l'uno l'opposto dell'altro. Tuttavia, negli scenari reali, un robot si trova spesso in un ambiente completamente sconosciuto, senza avere a disposizione né una mappa preesistente né una stima esatta della propria posizione. È in questa situazione che entra in gioco un terzo e più complesso problema, che consiste nell'unione dei due precedenti, lo *SLAM* (Simultaneous Localization and Mapping).

2 SLAM

Lo SLAM è il processo computazionale che permette a un veicolo o a un robot di costruire una mappa di un ambiente sconosciuto mentre, contemporaneamente, calcola la propria posizione all'interno di quella stessa mappa in fase di costruzione.

Questo problema viene spesso descritto come un paradosso, per costruire una mappa precisa è necessario conoscere l'esatta posizione del veicolo, ma per stimare l'esatta posizione del veicolo serve avere una mappa di riferimento.

Lo SLAM risolve questo paradosso fondendo in tempo reale i dati provenienti da vari sensori (come telecamere, LiDAR, radar e odometria) attraverso algoritmi probabilistici (come i Filtri di Kalman o i Particle Filter). In questo modo, ogni volta che il veicolo acquisisce nuove informazioni sull'ambiente, il sistema aggiorna simultaneamente sia la geometria della mappa sia la stima della posizione del veicolo al suo interno.

3 Filtro di Kalman

Il *Filtro di Kalman* è un algoritmo di stima ottimale che opera in modo ricorsivo in tempo reale. Il suo scopo è stimare lo *stato* di un sistema dinamico a partire da una serie di misurazioni sensoriali che sono intrinsecamente incomplete e affette da rumore.

Il funzionamento del filtro si basa su un ciclo continuo diviso in due fasi principali:

- *Predizione*, utilizzando un modello matematico del movimento, il filtro *predice* lo stato successivo del robot e calcola il livello di incertezza di questa previsione.

- *Aggiornamento*, il filtro confronta il dato reale con quello ottenuto dalla previsione. Attraverso un coefficiente matematico chiamato Guadagno di Kalman, il filtro stabilisce quanta fiducia dare alla predizione teorica e quanta alla misurazione reale, combinandole per ottenere la stima finale.

Il Filtro di Kalman standard presuppone però un vincolo matematico molto rigido: il sistema deve essere lineare e il rumore deve seguire una distribuzione gaussiana.

3.1 Filtro di Kalman esteso

Nella robotica reale, la cinematica dei veicoli (come un unicycle) e i modelli dei sensori sono governati da leggi fortemente non lineari. Per questo è stata sviluppata una variante del filtro chiamata *EKF* (Extended Kalman Filter), che permette di propagare le covarianze e calcolare il guadagno utilizzando modelli linearizzati.

L'idea fondamentale alla base dell'EKF è l'uso dell'espansione in Serie di Taylor di primo ordine calcolata attorno alla stima corrente dello stato. In pratica, l'algoritmo calcola le derivate parziali delle funzioni non lineari rispetto alle variabili di stato e di rumore. Le matrici che contengono queste derivate parziali prendono il nome di Matrici Jacobiane.

Il ciclo dell'EKF segue questa struttura matematica rigorosa:

Come detto per il *Filtro di Kalman Classico* anche l'EKF segue la stessa struttura base:

Fase di Predizione Lo stato futuro del robot viene predetto usando le equazioni non lineari del moto (f_k), mentre l'incertezza viene propagata usando le Jacobiane del modello cinematico rispetto allo stato (A) e rispetto al rumore di processo (G).

$$\begin{cases} \hat{x}(k+1)^- = f_k(\hat{x}(k), u(k)) \\ P(k+1)^- = A(k)P(k)A(k)^T + G(k)Q(k)G(k)^T \end{cases}$$

Fase di Aggiornamento Si calcola la Jacobiana del modello di misura (H). Successivamente si valuta la *covarianza dei residui* (S) e il Guadagno di Kalman (W) per correggere lo stato usando l'innovazione, ovvero la differenza tra la misura reale $z(k+1)$ e la misura predetta non lineare $h_{k+1}(\hat{x}(k+1)^-)$.

$$\begin{cases} S(k+1) = H(k+1)P(k+1)^-H(k+1)^T + R(k+1) \\ W(k+1) = P(k+1)^-H(k+1)^T S(k+1)^{-1} \\ \hat{x}(k+1) = \hat{x}(k+1)^- + W(k+1)(z(k+1) - h_{k+1}(\hat{x}(k+1)^-)) \\ P(k+1) = (I - W(k+1)H(k+1))P(k+1)^- \end{cases}$$

3.2 Modello Cinematico dell'Uniciclo

Il veicolo che viene usato per la simulazione segue il modello cinematico dell'*uniciclo* che rappresenta un classico esempio di un sistema robotico non lineare e non olonomo.

Definizione dello stato

Il modello dell'uniciclo rappresenta un veicolo che si muove su un piano bidimensionale (2D). Lo stato cinematico completo del sistema in un generico istante di tempo continuo t è descritto da un vettore di stato a 4 dimensioni:

$$S = \begin{bmatrix} x \\ y \\ v \\ \theta \end{bmatrix}$$

dove $x(t)$ e $y(t)$ sono le coordinate cartesiane del centro di istantanea rotazione del robot rispetto a un sistema di riferimento globale fisso, $v(t)$ è la velocità lineare di avanzamento e $\theta(t)$ è l'angolo di imbardata (heading), ovvero l'orientamento del veicolo rispetto all'asse x globale.

Gli ingressi di controllo (ovvero le forze motrici virtuali) del sistema sono descritti dal vettore:

$$u(t) = \begin{bmatrix} a(t) \\ \omega(t) \end{bmatrix}$$

dove $a(t)$ è l'accelerazione lineare e $\omega(t)$ la velocità angolare.

Modello cinematico

Sfruttando il vincolo appena descritto, possiamo scomporre la velocità lineare $v(t)$ sugli assi cartesiani globali. Questo ci permette di scrivere il sistema di Equazioni Differenziali Ordinarie (ODE) che governa l'evoluzione analitica del sistema nel tempo:

$$\dot{x}(t) = f(x(t), u(t)) \rightarrow \begin{cases} \dot{x} = v \cos(\theta) \\ \dot{y} = v \sin(\theta) \\ \dot{v} = a \\ \dot{\theta} = \omega \end{cases}$$

Da questo sistema si riesce chiaramente a vedere la componente non lineare ed questa è il motivo per cui non si può usare il *Filtro di Kalman Standard* ma invece bisogna usare la versione estesa del filtro il quale sfrutta le serie di Taylor per lineare queste funzioni.

4 Modelli dei Sensori

Nel simulatore, l'uniciclo è equipaggiato con due sensori: - *IMU* - *Telecamera* questa coppia rappresenta una configurazione molto comune essendo che solitamente si tende ad accoppiare sensori propriocettivi con sensori eterocettivi per ragioni che verranno trattate in seguito.

4.1 IMU Planare

L'IMU è un sensore propriocettivo che misura accelerazioni, nel caso della versione planare le misurazioni ottenute sono:

- Accelerazione lineare lungo l'asse x del robot (a_x).
- Accelerazione lineare lungo l'asse y del robot (a_y).
- Velocità angolare attorno all'asse z (giroscopio, ω).

Modello del rumore e implementazione

I sensori reali sono affetti da imperfezioni, quindi nella classe IMU della simulazione al segnale generato dal calcolo della cinematica dell'uniciclo vengono aggiunti due tipi di disturbo:

- **Rumore Gaussiano** fluttuazioni casuali a media nulla aggiunte a ogni singola lettura
- **Bias** un errore sistematico a lenta variazione temporale. Nel codice, il bias è modellato come un *Random Walk*, ovvero il bias attuale è uguale al bias precedente più un piccolo rumore gaussiano.

```
# Update bias random walk
self.bg += delta_t * self.rng.normal(scale=self.gyro_bias_rw_std)
self.ba += delta_t * self.rng.normal(scale=self.accel_bias_rw_std, size=2)

# ...

# Misure ottenute dall'IMU con aggiunta di bias e rumore bianco
gyro_z = (omega + self.bg) + self.rng.normal(scale=self.gyro_noise_std)
accel_b = (ab + self.ba) + self.rng.normal(scale=self.accel_noise_std, size=2)
```

Problemi con sensori propriocettivi

Conoscendo l'accelerazione di ogni asse del robot è teoricamente ricavabile lo stato del robot semplicemente integrando l'accelerazione e poi integrando la velocità. Tuttavia non viene mai

usata una IMU o piu' in generale un sensore propriocettivo, come unico sensore, perche' si puo' dimostrare che questi tipi di sensori accumulano errore.

Quindi per il caso della *IMU*:

$$x(t) = x_0 + v_0 t + \frac{1}{2} a_m t^2 = x_0 + v_0 t + \frac{1}{2} a t^2 + \frac{1}{2} \epsilon t^2$$

$$E(x(t)) = E(\bar{x}(t) + \frac{1}{2} \epsilon t^2) = E(\bar{x}(t)) + E(\frac{1}{2} t^2 E(\epsilon)) \quad (1)$$

$$= \bar{x}(t) + \frac{1}{2} t^2 E(\epsilon) \quad (2)$$

$$E((x(t) - \bar{x}(t))^2) = E((\frac{1}{2} \epsilon t^2)^2) = \frac{1}{4} t^4 E(\epsilon^2) \quad (3)$$

$$= \frac{1}{4} t^4 E((\epsilon - E(\epsilon))^2) \quad (4)$$

$$= \frac{1}{4} t^4 \sigma^2 \quad (5)$$

$$Sd \rightarrow \sigma_x = \frac{t^2}{2} \sigma$$

Quindi possiamo vedere chiaramente che la deviazione standard (*sd*) del sensore dipende direttamente dal tempo.

4.2 Telecamera (Modello Pinhole)

La telecamera funge da sensore esterolettivo per riconoscere elementi dell'ambiente (i *landmark*) la cui posizione nel frame world $\langle w \rangle$ e' conosciuta.

Funzionamento e Vincoli Visivi

La classe **Camera** in `unicycle_estimation.py` simula un modello ottico di tipo *pinhole* (foro stenopeico) monodimensionale. Il sensore non vede in tutte le direzioni in modo illimitato, ma è vincolato da: - **Field of View (FOV)**: un angolo visivo limitato (impostato a 45 gradi). - **Range**: una distanza minima (`min_range = 1.0`) e massima (`max_range = 10.0`). Il metodo `check_fov()` si assicura che solo i landmark che rientrano in queste limitazioni geometriche generino una misurazione valida.

Proiezione e Utilizzo nell'EKF (Fase di Aggiornamento)

Quando un landmark è visibile, la sua posizione reale nel mondo (l_w) viene trasformata nelle coordinate locali del robot (Body frame, l_b) tramite la matrice di rotazione inversa R_{BW} . Successivamente, per simulare la formazione dell'immagine sul sensore ottico, le coordinate vengono normalizzate rispetto alla profondità x (coordinate omogenee) e moltiplicate per i parametri intrinseci della telecamera, ottenendo una coordinata in pixel u affetta da rumore gaussiano.

Nel file `kalman_filter.py`, la fase di *Aggiornamento* utilizza queste letture in pixel per correggere lo stato. Il metodo `measurement_model()` replica esattamente la matematica della proiezione ottica per calcolare la misura attesa $\hat{z}$. Nello stesso momento, la funzione `measurement_jacobian()` calcola la matrice H , applicando la regola della catena delle derivate (Chain Rule):

$$H = \frac{\partial u}{\partial \tilde{l}_b} \cdot \frac{\partial \tilde{l}_b}{\partial l_b} \cdot \frac{\partial l_b}{\partial x}$$

Questa matrice H è fondamentale per calcolare l'Incertezza dell'Innovazione S e il Guadagno di Kalman W , chiudendo così il ciclo dell'algoritmo ricorsivo e permettendo al sistema di gestire gli outlier tramite i calcoli della Distanza di Mahalanobis.

4.3 Gestione degli Outliers

I filtri di Kalman, pur essendo stimatori ottimali, sono per natura estremamente sensibili alle misurazioni anomale (outlier). Un singolo dato fortemente errato introdotto nella fase di aggiornamento può corrompere irrimediabilmente la stima dello stato e le matrici di covarianza, portando il filtro a divergere. È quindi fondamentale implementare un meccanismo di *gating* (un cancello logico) che validi statisticamente i dati prima di processarli.

La Distanza di Mahalanobis

Per risolvere questo problema, il filtro degli outlier si basa sulla **Distanza di Mahalanobis**. Si tratta di una misura statistica che valuta quanto un dato si allontani da una distribuzione attesa, *pesando* tale distanza rispetto all'incertezza (covarianza) del sistema stesso.

Matematicamente, data una misurazione z , un valore medio atteso μ e una matrice di covarianza Σ che descrive l'incertezza, la distanza di Mahalanobis al quadrato D^2 è definita come:

$$D^2 = (z - \mu)^T \Sigma^{-1} (z - \mu)$$

Da queste equazioni si evince il comportamento “adattivo” del filtro: - Se l’incertezza Σ è piccola (matrice “stretta”), anche un piccolo scostamento ($z - \mu$) genererà un D^2 molto grande, rendendo il filtro severo. - Se l’incertezza Σ è grande, il termine Σ^{-1} “ammorbidisce” l’impatto dell’errore, rendendo il filtro più tollerante.

4.4 Il Test del Chi-Quadro (χ^2)

Una volta calcolato il valore D^2 , come stabiliamo se è “troppo grande”? Sotto l’assunzione che il rumore del sistema abbia una distribuzione normale (Gaussiana), la variabile statistica D^2 segue una distribuzione del **Chi-Quadro** (χ^2) con un numero di gradi di libertà pari alla dimensionalità della misurazione.

Il filtro degli outlier funziona impostando un **livello di confidenza** (es. 95% o 99%). Tramite le tabelle statistiche della distribuzione χ^2 , si ricava un valore di soglia. Il processo decisionale è quindi un semplice test d’ipotesi:

1. Se $D^2 \leq$ soglia: Il dato appartiene statisticamente alla distribuzione attesa. Viene accettato.
2. Se $D^2 >$ soglia: Il dato è statisticamente troppo improbabile. Viene classificato come outlier e rigettato.